

Software Requirements Specification

ScreenIt

Resume Screening & Candidate Ranking System

Prepared for: **PSPC**

Prepared by: IT interns

Version 1.0 – Draft

August 3, 2026

Internal / Confidential – For distribution within PSPC only

Table of Contents

Table of Contents.....	2
Revision History.....	5
1. Introduction.....	6
1.1 Purpose.....	6
1.2 Document Conventions.....	6
1.3 Intended Audience and Reading Suggestions.....	6
1.4 Project Scope.....	6
1.5 References.....	7
2. Overall Description.....	8
2.1 Product Perspective.....	8
2.2 Product Features.....	8
2.3 User Classes and Characteristics.....	8
2.4 Operating Environment.....	9
2.5 Design and Implementation Constraints.....	9
2.6 User Documentation.....	9
2.7 Assumptions and Dependencies.....	9
3. System Features.....	10
3.1 Resume Ingestion & Parsing.....	10
Description and Priority.....	10
Stimulus/Response Sequence.....	10
Functional Requirements.....	10
3.2 Candidate Field Extraction.....	10
Description and Priority.....	10
Stimulus/Response Sequence.....	10
Functional Requirements.....	10
3.3 Policy-Based Eligibility Screening.....	11
Description and Priority.....	11
Stimulus/Response Sequence.....	11
Functional Requirements.....	11
3.4 Candidate Ranking.....	11
Description and Priority.....	11
Stimulus/Response Sequence.....	11
Functional Requirements.....	12
3.5 Job Description Management.....	12
Description and Priority.....	12
Stimulus/Response Sequence.....	12
Functional Requirements.....	12

3.6 Policy Management.....	12
Description and Priority.....	12
Stimulus/Response Sequence.....	12
Functional Requirements.....	12
3.7 Candidate Search & Browse.....	13
Description and Priority.....	13
Stimulus/Response Sequence.....	13
Functional Requirements.....	13
3.8 Reporting & Export.....	13
Description and Priority.....	13
Stimulus/Response Sequence.....	13
Functional Requirements.....	13
3.9 Dashboard & Analytics.....	13
Description and Priority.....	13
Stimulus/Response Sequence.....	13
Functional Requirements.....	13
4. External Interface Requirements.....	14
4.1 User Interfaces.....	14
4.2 Hardware Interfaces.....	14
4.3 Software Interfaces.....	14
4.4 Communications Interfaces.....	15
5. Other Nonfunctional Requirements.....	16
5.1 Performance Requirements.....	16
5.2 Safety Requirements.....	16
5.3 Security Requirements.....	16
5.4 Software Quality Attributes.....	16
6. Other Requirements.....	18
7. Future Enhancements.....	19
7.1 Internal, IP-Restricted Deployment.....	19
7.2 Integration with PSPC's Application Website.....	19
7.3 Web-Based Policy Management.....	19
7.4 Editable Job Description Management.....	19
7.5 Phase Tracking and Per-Phase Reporting.....	19
7.6 Excel-Based Screening.....	19
7.7 Test-Score Ranking Signal.....	19
7.8 Authentication and Authorization.....	19
7.9 Multi-User / Concurrent Run Support.....	20
Appendix A: Glossary.....	21

Appendix B: Analysis Models.....	22
Pipeline Flow.....	22
Appendix C: Issues List.....	23

Revision History

Name	Date	Reason For Changes	Version
Maheen Fatima –IT intern <author>	Aug 3, 2026	To specify the working requirements of the personalized ATS system and possible future additions	1.0

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements of ScreenIt, a resume screening and candidate ranking system built for the Human Resources department of PSPC. It covers Version 1.0 of the system: the ingestion, extraction, eligibility screening, ranking, and reporting pipeline currently implemented and running, along with the React-based web interface used to operate it. Where a capability is planned but not yet built, it is explicitly marked as a future requirement rather than described as current behavior.

1.2 Document Conventions

Each functional requirement is uniquely tagged (e.g. REQ-1, REQ-2, ...) within its System Feature section for traceability. "Shall" denotes a mandatory requirement; "should" denotes a desirable but non-mandatory one. Section 7 (Future Enhancements) and Appendix C (Issues List) separate proposed and not-yet-built work from the current, implemented functionality described in Sections 3 and 4. Text in square brackets, e.g. [Author / IT Team Name], is a placeholder to be replaced with an actual value before this document is finalized or distributed.

1.3 Intended Audience and Reading Suggestions

This document is intended for four audiences: (1) HR staff who will use ScreenIt day-to-day to screen and rank candidates; (2) the IT/development team responsible for maintaining, extending, and eventually deploying the system beyond localhost; (3) HR and IT management sponsoring the project; and (4) any future maintainers or auditors reviewing what the system does and does not currently do. Readers new to the system should read Section 2 (Overall Description) first for context, then Section 3 (System Features) for the detail relevant to their area. Developers extending the pipeline should also read Section 4 (External Interface Requirements) for the current API contract, and Section 5.3 (Security Requirements) before any deployment beyond a single local machine.

1.4 Project Scope

HR at PSPC currently screens candidates by manually comparing resumes against each job description and a set of eligibility rules (age, education, experience, and similar policy checks). ScreenIt automates this: HR points the system at a folder of resumes and a job description, and the system extracts each candidate's details, checks them against a job-specific eligibility policy, and ranks the eligible candidates by how well they match the job description, using a combination of keyword-based (BM25) and semantic (embedding-based) matching.

The current release covers the full pipeline from resume ingestion through ranked results, plus HR-facing forms to author job descriptions and eligibility policies, and a dashboard for reviewing outcomes. It currently runs on a single local machine (localhost) with no authentication layer. Deployment as a shared intranet service, integration with PSPC's main application-form website, in-browser policy editing, phased-report tracking, Excel-based

ingestion, and an additional test-score ranking signal are identified as future work in Section 7 (Future Enhancements), to be scoped and built by the IT team.

1.5 References

- backend/requirements.txt – Python dependency manifest for the current implementation.
- backend/main.py – FastAPI application defining the current REST API contract (Section 4.3).
- backend/src/policy_engine/eligibility.py – reference implementation of the eligibility rule schema described in Section 3.3.
- [XYZ HR System] – PSPC's existing HR platform, used as the visual/UX reference for ScreenIt's interface (replace with the actual system name).
- IEEE Std 830-1998 – Recommended Practice for Software Requirements Specifications (structural basis for this document).

2. Overall Description

2.1 Product Perspective

ScreenIt is a new, self-contained internal tool. It is not a replacement for PSPC's existing HR platform () but a specialized add-on that handles one workflow that platform does not: automated, policy-aware resume screening and ranking. ScreenIt's interface is deliberately styled to match [XYZ HR System]'s look and feel so that HR staff already familiar with that system have minimal relearning to do.

The system has two parts that run together: a Python/FastAPI backend that performs ingestion, extraction, eligibility evaluation, and ranking; and a React single-page frontend that HR uses to trigger pipeline runs, author job descriptions and policies, and review results. See Appendix B for a diagram of how these stages connect.

2.2 Product Features

At a high level, ScreenIt lets HR:

- Point the system at a folder of candidate resumes (and CNIC documents) and a job description file, and run the screening pipeline against them.
- Have candidates automatically checked against a job-specific eligibility policy (age, age relaxations, minimum education, minimum experience, supervisory experience, preferred skills).
- Have eligible candidates automatically ranked against the job description using a combined lexical (BM25) and semantic (sentence-embedding) match score.
- Choose how many top-ranked candidates ("Top-K") to review.
- View a dashboard of KPIs and charts summarizing the most recent run.
- Browse and search all processed candidates.
- Author new job description ads through a form, saved automatically as a formatted Word document.
- Author new eligibility policies through a form, saved automatically as a YAML rules file.
- Export candidate rankings for reporting.

2.3 User Classes and Characteristics

User Class	Description	Frequency
HR Recruiter / Screener	Runs the pipeline against resumes and a job description, reviews rankings, searches/browses candidates, exports reports. Primary day-to-day user.	Daily
HR Policy Administrator	Authors and maintains job descriptions and eligibility policies through the respective forms.	Per new job opening
IT Administrator / Maintainer	Runs and maintains the backend and frontend, manages the configs/, ads/, and runs/ directories, extends the pipeline.	As needed
Hiring Manager (Future)	Read-only visibility into ranked candidates for roles they own. Not yet supported — no role-based access exists today.	Future

2.4 Operating Environment

Backend: Python 3, served by FastAPI on Uvicorn. Frontend: React 19 (Vite build), a single-page application accessed through a modern desktop browser (Chrome/Edge recommended). As of this version, both run on a single Windows workstation and communicate over localhost; the system is not yet accessible over the network. See Section 7.1 for the planned move to an internal, IP-restricted deployment.

2.5 Design and Implementation Constraints

- Legacy .doc files are read via pywin32 (Windows COM automation), which only works on Windows and requires Microsoft Word to be installed on the host machine; this is a known point of fragility (see the ingestion error handling in REQ-5).
- OCR accuracy (pytesseract / rapidocr-onnxruntime) depends on scan/image quality and is a fallback path, not the primary extraction method.
- NLP field extraction (spaCy) currently assumes English-language resumes.
- Candidate PII, including CNIC numbers, must remain within the organization's own network; the system does not transmit data to any external service.
- Pipeline state for GET endpoints (dashboard, ranking, queue) is held in a single in-memory variable on the backend, meaning only the most recently completed run is available at a time. There is currently no persistent database and no support for multiple concurrent HR users running separate jobs at once.
- Job description and resume folder locations are currently either typed/pasted server paths or files uploaded from the user's own machine; there is no integration yet with any external system that supplies these automatically.

2.6 User Documentation

No standalone user manual exists yet. In-app hints (field-level help text on the Job Description, Policy, and Resume Screening forms) currently serve this purpose. A short HR user guide and an IT setup/runbook are recommended future documentation deliverables, not yet scheduled.

2.7 Assumptions and Dependencies

- Assumes candidate resumes are predominantly in English; multilingual support is not in scope for this version.
- Assumes resumes are organized on disk (or uploaded) following the <job_category>/<candidate_id>/{resume, cnic} folder convention.
- Depends on third-party libraries and models: spaCy language models, sentence-transformers pretrained embedding models, FAISS, rank-bm25, and a locally installed Tesseract OCR binary.
- Assumes the machine running the backend has network access limited to PSPC's trusted intranet; no authentication layer is currently assumed to protect the API.
- Assumes each job category maps to exactly one active eligibility policy YAML file at a time.

3. System Features

Requirements below are organized by pipeline stage and by user-facing module, matching the system's actual module boundaries (backend/src/ subpackages and frontend pages).

3.1 Resume Ingestion & Parsing

Description and Priority

Priority: **High**

Stimulus/Response Sequence

HR provides a folder path (or uploads a folder) containing one subfolder per candidate, each with a resume file and, where available, a CNIC document. The system reads every file, extracts raw text (using OCR where needed), corrects out-of-order text blocks, and writes one structured record per candidate for the next stage.

Functional Requirements

REQ-1: The system shall accept a resumes folder following the structure `<job_category>/<candidate_id>/{resume, cnic}`, either as a server-side path or as files uploaded from the user's own machine.

REQ-2: The system shall support resumes in PDF, DOC, and DOCX formats.

REQ-3: The system shall apply OCR (pytesseract, with rapidocr-onnxruntime as an alternate engine) to image-based or scanned pages when direct text extraction does not yield sufficient content.

REQ-4: The system shall detect and correct scrambled or out-of-order text extracted from PDF resumes ("descrambling") before passing content to field extraction.

REQ-5: The system shall isolate errors to the affected candidate (e.g. an unreadable legacy .doc file) and continue processing the remaining candidates in the batch rather than aborting the run.

REQ-6: The system shall write ingestion output as a structured JSON file for the run (01_ingested.json).

3.2 Candidate Field Extraction

Description and Priority

Priority: **High**

Stimulus/Response Sequence

The system parses each candidate's ingested text into structured fields using NLP-based extraction, producing the profile data used by both eligibility screening and ranking.

Functional Requirements

REQ-7: The system shall extract personal information: name, contact details, date of birth, CNIC number, address/province, gender, and nationality, where present in the resume.

REQ-8: The system shall extract education history, including degree name and degree level per entry.

REQ-9: The system shall extract experience history, including total years of experience and years of management/supervisory experience.

REQ-10: The system shall extract a list of skills and certifications mentioned in the resume.

REQ-11: The system shall flag candidate records with missing required fields or low-confidence extraction for manual review, rather than silently guessing.

REQ-12: The system shall use fuzzy string matching (RapidFuzz) to normalize degree and skill names against known reference lists.

3.3 Policy-Based Eligibility Screening

Description and Priority

Priority: **High**

Stimulus/Response Sequence

Each extracted candidate is evaluated against the eligibility policy YAML file for their job category. The result (Eligible / Not Eligible), with a reason per check, is attached to the candidate before ranking.

Functional Requirements

REQ-13: The system shall load the eligibility policy YAML file matching the candidate's job category before evaluation.

REQ-14: The system shall calculate a candidate's age from their date of birth relative to a policy-configured reference date.

REQ-15: The system shall apply configurable age relaxations for regional candidates, current PSPC employees applying internally, and candidates with qualifying security-printing experience, as defined per policy.

REQ-16: The system shall verify the candidate meets the policy's minimum education level and, where specified, that their degree name matches one of the policy's accepted degrees (or that the policy accepts an equivalent-or-higher qualification).

REQ-17: The system shall verify the candidate meets the policy's minimum years of work experience and, where required by the policy, minimum years of supervisory experience.

REQ-18: The system shall record which of the policy's preferred (non-mandatory) skills each candidate has, without affecting eligibility status.

REQ-19: The system shall mark a candidate "Not Eligible" if any mandatory check (education, experience, supervisory, or age) fails, and shall record a human-readable reason for each check performed.

REQ-20: The system shall exclude "Not Eligible" candidates from the ranking stage.

3.4 Candidate Ranking

Description and Priority

Priority: **High**

Stimulus/Response Sequence

Eligible candidates are ranked against the supplied job description using a hybrid lexical and semantic matching score, and the top results are returned to the frontend.

Functional Requirements

REQ-21: The system shall accept a job description file (DOCX or TXT) as ranking input.

REQ-22: The system shall compute a BM25-based lexical relevance score between each eligible candidate's profile and the job description.

REQ-23: The system shall compute a semantic similarity score using sentence embeddings and FAISS-based similarity search.

REQ-24: The system shall combine the lexical and semantic scores into a single final match score per candidate.

REQ-25: The system shall allow HR to configure Top-K, the number of top-ranked candidates returned, per run.

REQ-26: The system shall report matched required skills, missing required skills, and matched preferred skills for each ranked candidate.

3.5 Job Description Management

Description and Priority

Priority: **Medium**

Stimulus/Response Sequence

HR authors a new job description through a form; the system generates a formatted Word document from it.

Functional Requirements

REQ-27: The system shall provide a form to capture job title, department, location, employment type (Permanent, Outsourced, Contractual, Internship, or Apprenticeship), a free-text experience requirement, a free-text education requirement, required skills, preferred skills, and a bulleted list of responsibilities.

REQ-28: The system shall generate a formatted .docx job ad from the submitted form and save it under the job-ads directory, named after the job title.

3.6 Policy Management

Description and Priority

Priority: **Medium**

Stimulus/Response Sequence

HR (or an HR policy administrator) authors a new eligibility policy through a form; the system generates a YAML rules file consumed by the eligibility engine (Section 3.3).

Functional Requirements

REQ-30: The system shall provide a form to capture: job name; a reference date for age calculation; maximum age; regional, internal-employee, and special-case age relaxations; minimum education level and accepted degree names (or an equivalent-or-higher-qualification toggle); minimum experience and, optionally, minimum supervisory experience; and preferred skills.

REQ-31: The system shall generate a policy YAML file from the submitted form and save it under the policy configuration directory, named after the job.

3.7 Candidate Search & Browse

Description and Priority

Priority: **Medium**

Stimulus/Response Sequence

HR browses all candidates from the most recent run and can search across them.

Functional Requirements

REQ-33: The system shall list all candidates processed in the most recently completed pipeline run.

REQ-34: The system shall allow free-text search across candidate name, skills, and job role.

REQ-35: The system shall display each candidate's eligibility outcome and, where available, their ranking score.

3.8 Reporting & Export

Description and Priority

Priority: **Medium**

Stimulus/Response Sequence

HR downloads results for record-keeping or sharing outside the system.

Functional Requirements

REQ-36: The system shall allow HR to export the current candidate ranking to an Excel (.xlsx) file.

3.9 Dashboard & Analytics

Description and Priority

Priority: **Medium**

Stimulus/Response Sequence

HR reviews a summary of the most recent run without needing to open individual candidate records.

Functional Requirements

REQ-38: The system shall display summary KPIs: total resumes processed, number screened, number shortlisted, and average match score.

REQ-39: The system shall display distribution charts for education level, top skills, and experience-years buckets across processed candidates.

REQ-40: The system shall display a table of recent screenings with each candidate's policy outcome and match score.

4. External Interface Requirements

4.1 User Interfaces

ScreenIt's frontend is a React single-page application with a persistent left sidebar (Dashboard, Resume Screening, Candidate Ranking, Candidates, Job Descriptions, Analytics, Reports, Policy, About) and a top search bar. Its visual style — color palette, card-based layout, and typography — is intentionally modeled on [XYZ HR System] so that HR staff already familiar with that platform have minimal relearning to do. Forms (Resume Screening, Job Description, Policy) follow a consistent pattern: labeled fields grouped into sections, inline hints explaining format and optionality, chip-style inputs for repeatable values (skills, degrees, regions), and a status message confirming success or surfacing an error after submission.

4.2 Hardware Interfaces

No specialized hardware is required. The system runs on a standard office workstation; CPU-based inference is sufficient for current candidate volumes using the sentence-transformers and FAISS libraries. A GPU is not required for this version. Scanning/printing hardware is out of scope — scanned documents are expected to already exist as image or PDF files when ingested.

4.3 Software Interfaces

The backend exposes a REST API (FastAPI, served by Uvicorn) that the frontend consumes over HTTP. The current endpoints are:

Method & Path	Purpose
POST /api/pipeline/run	Runs ingestion → extraction → eligibility → (optional) ranking for a given resumes folder and job description.
POST /api/candidates/process	Runs the full pipeline (ranking required) and returns one merged record per candidate.
GET /api/dashboard/summary	Returns KPIs, pipeline funnel, and chart data for the most recent run.
GET /api/ranking/latest	Returns the ranked candidate list for the most recent run.
GET /api/screening/queue	Returns per-file ingestion/extraction status for the most recent run.
GET /api/health	Backend and ranking-stage health check.
POST /api/policy/save	Saves a generated eligibility policy YAML to the configuration directory.
POST /api/jobs/save	Saves a generated job description as a .docx file to the job-ads directory.
POST /api/upload/resumes	Uploads a folder of resumes from the user's own machine for processing.
POST /api/upload/job-description	Uploads a single job description file from the user's own machine.

Supporting software dependencies include: PyMuPDF and python-docx for document parsing; pytesseract and rapidocr-onnxruntime for OCR; spaCy and RapidFuzz for NLP and fuzzy matching; rank-bm25 and sentence-transformers for ranking; FAISS for similarity search; PyYAML for policy files; and openpyxl/xlsxwriter for report export. See `backend/requirements.txt` for the complete, versioned list.

4.4 Communications Interfaces

The frontend and backend currently communicate over plain HTTP on localhost, with JSON request/response bodies for most endpoints and multipart/form-data for file uploads. Cross-Origin Resource Sharing (CORS) is currently configured to allow all origins, which is acceptable only for local development and must be restricted before any shared deployment (see Section 5.3). There is no message queue, email, or external network integration in the current version.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

Pipeline runtime scales with the number of candidates and the proportion requiring OCR. As a working target for the current single-workstation deployment: a batch of approximately 50 resumes should complete ingestion through ranking within a few minutes on standard office hardware. This target has not yet been formally benchmarked and should be validated once representative volumes are available. Once eligibility screening has completed, ranking a Top-K result set should complete within seconds.

5.2 Safety Requirements

ScreenIt is not a safety-critical system. The relevant risk is incorrect or silently-wrong data influencing an HR hiring decision. Accordingly, the system must flag records it cannot confidently extract or evaluate (REQ-11, REQ-19) rather than guessing, and eligibility reasons must always be shown alongside the Eligible/Not Eligible outcome so HR can verify the system's reasoning rather than trusting it blindly.

5.3 Security Requirements

Current gap: as of this version, the system has no authentication or authorization layer, and the backend's CORS policy permits requests from any origin. File upload and pipeline-run endpoints are unauthenticated. This is acceptable only while the system runs on a single machine used by a trusted individual, and must be addressed before the “internal system, accessible within HR IPs” deployment described in Section 7.1 — at minimum via network-level IP restriction, and longer-term via user authentication (Section 7.8).

Candidate personal data, including CNIC numbers, must remain within PSPC's own network at all times; the system does not call any external API and must continue not to do so without an explicit data-handling review. Any future integration with an external website or service (Section 7.2) must be reviewed against this constraint before implementation.

5.4 Software Quality Attributes

Attribute	How this version addresses it
Usability	Form-first workflows styled after the HR team's existing, familiar [XYZ HR System] interface; inline hints on every form field.
Maintainability	Pipeline stages are separated into distinct modules (src/ingestion, src/extraction, src/policy_engine, src/retrieval) with a documented API contract in main.py.
Reliability	Per-candidate error isolation (REQ-5) so a single unreadable file does not halt a batch run.
Portability	Backend and frontend are cross-platform (Python/FastAPI, React), with one known exception: legacy .doc parsing depends on pywin32 and is Windows-only.
Traceability	Every functional requirement in this document is tagged (REQ-N) for reference in design, test, and defect-tracking artifacts.

6. Other Requirements

- Data retention: pipeline run outputs are currently written under backend/runs/<run_id>/ with no automatic cleanup or retention policy. A retention/purge policy should be defined before production use (Appendix C).
- Localization: the current version supports English-language resumes and an English-language interface only. Urdu or other language support is out of scope for this version.
- Legal/compliance: candidate records include CNIC numbers and other personal data subject to PSPC's data protection obligations. PSPC's legal/compliance function should confirm any additional handling, storage, or retention requirements beyond what is stated in Section 5.3.
- Configuration data: eligibility policies (YAML) and job descriptions (.docx) are stored as files on the backend's local disk rather than in a database. This is acceptable for the current single-workstation deployment and should be revisited if the system moves to a shared or multi-user deployment.

7. Future Enhancements

Everything above (Sections 1–6) describes ScreenIt Version 1.0 as it exists today. The items in this section are proposed enhancements only — none of them are implemented in the current version. They are listed here, separately from the requirements above, so that what has already been built can be reviewed and signed off on its own merits, independent of this roadmap. Each item is tagged (FUT-N) for reference and should be scoped into its own requirements/design work by the PSPC IT team before development begins.

7.1 Internal, IP-Restricted Deployment

FUT-1: Deploy ScreenIt as a shared internal service accessible only from within PSPC's HR IP range, replacing the current single-workstation, localhost-only setup described in Section 2.4.

7.2 Integration with PSPC's Application Website

FUT-2: Automatically supply the job description (and potentially candidate submissions) from the forms already filled out on PSPC's main website, replacing the manually typed or uploaded path described in Sections 3.1 and 4.3.

7.3 Web-Based Policy Management

FUT-3: Allow HR to view, edit, and delete existing eligibility policies directly from the web interface, instead of only creating new ones through the Policy form (Section 3.6).

7.4 Editable Job Description Management

FUT-4: Allow HR to view, edit, and delete existing job description ads directly from the web interface, instead of only creating new ones through the Job Description form (Section 3.5).

7.5 Phase Tracking and Per-Phase Reporting

FUT-5: Track each candidate through discrete phases — form data collected, screened, ranked — and allow HR to download a report for any individual phase, not only the final ranking (extends Section 3.8).

7.6 Excel-Based Screening

FUT-6: Support ingesting candidate data directly from Excel spreadsheets as an alternative to — and potentially an eventual replacement for — the current document/text-based ingestion module (Section 3.1).

7.7 Test-Score Ranking Signal

FUT-7: Incorporate assessment/test batch results (e.g. PLIP batch scores) as an additional ranking signal alongside the existing BM25 and semantic scores described in Section 3.4.

7.8 Authentication and Authorization

FUT-8: Add a login/access-control layer so the system is no longer open to any user who can reach it on the network (see the current gap noted in Section 5.3). This is a prerequisite for FUT-1.

7.9 Multi-User / Concurrent Run Support

FUT-9: Move beyond the current single in-memory “latest run” model (Section 2.5) to support multiple HR users running separate pipeline jobs at the same time, most likely backed by a persistent database instead of per-run JSON files.

Appendix A: Glossary

Term	Definition
ATS	Applicant Tracking System — software used to manage the candidate recruitment and screening workflow.
BM25	Best Matching 25 — a keyword-based (lexical) relevance ranking algorithm used to score how well a candidate's text matches a job description.
CNIC	Computerized National Identity Card — the Pakistani national identity document; a source of candidate identity and date-of-birth data.
Descrambling	Correcting text that was extracted from a PDF in the wrong reading order, before it is passed to field extraction.
Eligibility Policy	A job-specific YAML rules file (e.g. age limits, education, experience) that a candidate is checked against before ranking.
FAISS	Facebook AI Similarity Search — a library used for fast nearest-neighbor search over semantic embeddings.
JD	Job Description — the document describing a role's requirements, used as the basis for ranking.
OCR	Optical Character Recognition — extracting text from scanned or image-based documents.
PSPC Employee Relaxation	A policy-configurable age relaxation applied to candidates who are already PSPC employees applying internally for a different position.
Semantic Score	A similarity score between a candidate's profile and a job description, computed from sentence embeddings rather than keyword overlap.
Top-K	The number of top-ranked candidates HR chooses to have returned from a given pipeline run.
YAML	A human-readable data format ("YAML Ain't Markup Language") used to store eligibility policies.

Appendix B: Analysis Models

Pipeline Flow

The backend pipeline runs as a linear sequence of stages, each writing its output as a JSON file for the next stage to consume:

Stage	Input	Output
1. Ingestion	Resumes folder (path or upload) + CNIC documents	01_ingested.json — raw text per candidate, OCR applied where needed, descrambled
2. Field Extraction	01_ingested.json	02_extracted.json — structured candidate profiles (personal info, education, experience, skills)
3. Eligibility Screening	02_extracted.json + job-specific policy YAML	03_eligibility.json — Eligible / Not Eligible + per-check reasons
4. Ranking	Eligible candidates + job description file	04_ranked.json — BM25 + semantic final score, Top-K
Frontend	Dashboard, Ranking, Candidates, Screening Queue endpoints	HR-facing views of the above outputs

Folder convention consumed by Stage 1:

- <resumes_folder>/<job_category>/<candidate_id>/resume.*
- <resumes_folder>/<job_category>/<candidate_id>/cnic.*

Job category (e.g. “senior_it_manager”) is the link between a candidate's folder, the job description used for ranking, and the eligibility policy YAML file selected in Stage 3.

Appendix C: Issues List

This list holds open decisions and unknowns — things that need an answer, not proposed features (those are in Section 7). Where an item relates to a specific future enhancement, its FUT-N tag is referenced.

ISSUE-1: Exact network/access-control mechanism for FUT-1 (IP allowlist, reverse proxy, or VPN-only access) is not yet decided.

ISSUE-2: API contract for the FUT-2 website integration (what data PSPC's application website would send, and how/when) is not yet defined.

ISSUE-3: Data retention/purge policy for stored run outputs, job description documents, and policy files (Section 6) is not yet defined.

ISSUE-4: Formal performance benchmarks (Section 5.1) have not yet been measured against representative candidate volumes.

ISSUE-5: Scope and expected data format for the FUT-6 Excel ingestion path is not yet defined.

ISSUE-6: Data source and scoring format for the FUT-7 PLIP batch test scores is not yet defined.